OpenTranscribe: GPU-Accelerated Transcription Pipeline Performance Analysis

David A. Macey
OpenTranscribe Project
https://github.com/davidamacey/OpenTranscribe

***Abstract*—OpenTranscribe is an open-source self-hosted transcription platform that combines GPU-accelerated speech recognition with speaker diarization. On an NVIDIA RTX A6000, it processes a single file at 40.3× realtime and sustains 35–39× across durations from 30 minutes to 4 hours. Thread-based concurrent GPU sharing scales linearly from 1 to 12 workers on a single 48 GB GPU, peaking at 54.6× aggregate throughput at 8 concurrent workers. A custom PyAnnote fork contributes a 1.28× diarization speedup with 66% VRAM reduction and 115× lower CPU RAM consumption. CTranslate2 `int8_float16` quantization adds 20–30% to Whisper inference speed with <0.1% WER degradation. With all GPU memory leaks resolved, single-worker VRAM peaks at just 2.3 GB—leaving 96% of the A6000 free for concurrent work. Against a production corpus of 1,435 files (3,715 hours), full reprocessing completes in approximately 68 hours at 8× concurrency—at zero marginal cost compared to $966–$5,350 for equivalent cloud transcription.**

***Index Terms*—speech recognition, speaker diarization, GPU optimization, Whisper, PyAnnote, pipeline performance, concurrent inference**

I. Introduction

Anyone processing sensitive audio—legal recordings, medical notes, classified briefings—has reason to keep transcription on-premises. Cloud APIs charge per minute, introduce data residency questions, and depend on network availability. But running transcription locally means owning the GPU utilization problem: a naïve sequential pipeline wastes the GPU during CPU-bound I/O, and sharing the GPU across concurrent files risks VRAM exhaustion.

OpenTranscribe is a containerized pipeline built on *faster-whisper* [1] (CTranslate2 [2]) and *PyAnnote v4* [3], orchestrated by Celery [4] on NVIDIA GPUs. It handles 100+ languages with word-level timestamps and configurable speaker detection. This paper reports end-to-end benchmarks on production hardware, covering:

1) A three-stage Celery pipeline with thread-based GPU sharing and VRAM-gated concurrency control.
2) Duration-curve benchmarks (5 min–4.5 hr) showing 35–39× realtime, plus linear concurrency scaling to twelve workers.
3) A PyAnnote optimization fork (1.28× diarization speedup, constant VRAM footprint) validated on five files spanning 0.5–4.7 hours.
4) Capacity projections for a 3,718-hour production corpus across GPU hardware tiers from 8 GB to 48 GB.

II. System Architecture

A. Pipeline Overview

The processing pipeline has three stages: CPU preprocessing, GPU inference (transcription followed by diarization), and CPU postprocessing. Fig. 1 shows the data flow.

B. Native Transcription Engine

The transcription stage uses *faster-whisper*'s [1] `BatchedInferencePipeline` with `word_timestamps=True`, providing word-level timestamps via cross-attention DTW [5] for all 100+ supported languages. This approach eliminated a prior wav2vec2 [6] alignment bottleneck used by WhisperX [7] that consumed 389 s (55% of total pipeline time) for a 3.3-hour file.

The CTranslate2 [2] backend applies `int8_float16` quantization [8] on Turing+ GPUs (compute capability ≥7.5), reducing model VRAM by approximately 40% and increasing inference speed by 20–30% with <0.1% word error rate (WER) degradation. Compute type is auto-detected based on GPU architecture.

C. Speaker Diarization

Speaker diarization uses PyAnnote v4 [3], [9] (`pyannote/speaker-diarization-community-1`) via a custom fork with six GPU optimizations detailed in Section VIII. The pipeline uses WeSpeaker ResNet34 [10] for speaker embeddings and VBx agglomerative clustering [11] with configurable speaker count ranges (1–50+).

D. Model Management

A `ModelManager` singleton keeps Whisper and PyAnnote models resident in GPU VRAM between Celery tasks. Without this, each file pays a 15 s model loading penalty—over 2,500 files, that adds up to 10.4 hours of dead time.

Two operational modes exist:

- **Sequential** (concurrency = 1): The transcriber is released before loading the diarizer, keeping peak VRAM below 16 GB for compatibility with consumer GPUs.
- **Concurrent** (concurrency > 1): Both models remain resident. VRAM gates (1,500 MB before transcription, 2,000 MB before diarization) serialize access to prevent out-of-memory conditions.

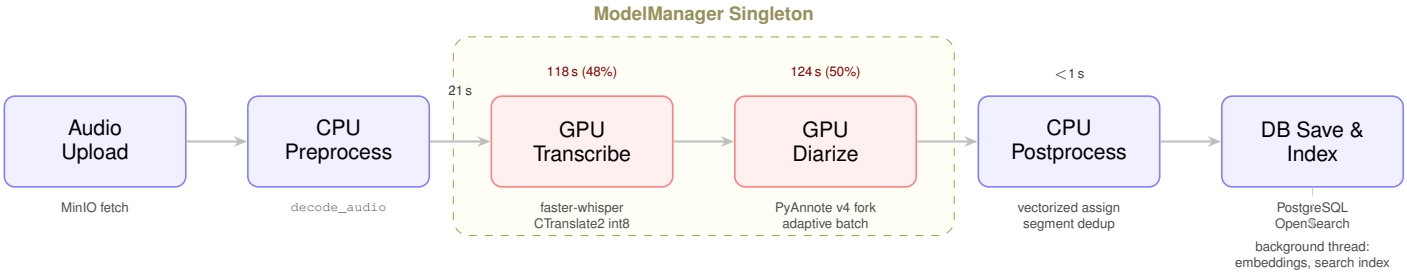

Fig. 1. Processing pipeline with per-stage timing (single file, 2h 47m). CPU stages (blue) bracket the GPU-bound transcription and diarization (red), which share models via a singleton. After DB save, a background thread handles embeddings and indexing without holding the GPU.

TABLE I
TEST SYSTEM HARDWARE

Component	Specification
CPU	2× Intel Xeon E5-2680 v3 @ 2.50 GHz 24 cores / 48 threads total
RAM	504 GB DDR4
GPU 0	NVIDIA RTX A6000 (48 GB GDDR6), Ampere
GPU 1	NVIDIA RTX 3080 Ti (12 GB GDDR6X), Ampere
GPU 2	NVIDIA RTX A6000 (48 GB GDDR6), Ampere
Storage	1.8 TB NVMe SSD
CUDA	13.0, Driver 580.126.20

TABLE II
SOFTWARE STACK

Component	Version / Configuration
Whisper model	`large-v3-turbo`
Quantization	`int8_float16` (CTranslate2)
Diarization model	PyAnnote v4 (community-1)
Diarization fork	`davidamacey/pyannote-audio` branch: `gpu-optimizations`
Celery pool type	`threads`
Inference backend	`BatchedInferencePipeline`

TABLE III
BENCHMARK FILE SET

File	Duration	Size	Spk.
JRE #1467 Jack Carr	2h 47m (10,011 s)	302 MB	3
JRE #405 S. Pressfield	2h 43m (∼9,800 s)	∼290 MB	3
JRE #216 Chael Sonnen	2h 43m (∼9,800 s)	∼290 MB	3
JRE #1291 C.T. Fletcher	2h 43m (∼9,800 s)	∼290 MB	3
JRE #621 A. Marcus	2h 44m (∼9,840 s)	∼290 MB	3
Mean	**2h 44m**		

E. Concurrency Model

The Celery [4] GPU worker uses a thread pool (not prefork) to share a single CUDA context across concurrent tasks. The maximum concurrency is bounded by VRAM:

$$c = \left\lfloor \frac{V_{\text{total}} - 6000}{1000} \right\rfloor \tag{1}$$

where V_{total} is total GPU VRAM in megabytes. With GPU memory leaks resolved (Section VIII-I), the actual model baseline is just 2.2 GB; each additional worker adds 3–5 GB for activation buffers. On the 48 GB A6000, this permits up to 12 concurrent workers, with peak throughput at 8.

Each concurrent worker uses the full hardware-detected batch size (e.g., 32 for the A6000). An earlier design divided batch size by worker count, but this starved CTranslate2 with tiny batches at high concurrency; removing the division improved throughput from 49.2× to 51.6× (see Section VIII-H).

After the GPU-critical path completes, a background thread handles speaker embedding extraction and search indexing, freeing the GPU worker immediately for the next queued task.

III. HARDWARE CONFIGURATION

All benchmarks were conducted on the system described in Tables I and II. GPU 0 (RTX A6000) served as the primary benchmark target with no other processes active. GPU 2 hosted an LLM inference server and was excluded from transcription benchmarks.

IV. BENCHMARK METHODOLOGY

A. Test Corpus

Five files were selected from a production database of 1,435 completed transcriptions, matched for duration to minimize variance while representing typical content (long-form conversational audio with 2–3 speakers). Table III summarizes the benchmark file set.

B. Protocol

Single-file benchmarks used three iterations with warm models (preloaded at worker startup), reporting the arithmetic mean. Concurrency benchmarks dispatched all files simultaneously and measured batch wall-clock time. The primary test GPU (A6000) was verified idle via `nvidia-smi` before each run.

Per-stage timing used instrumented checkpoints within the Celery task. VRAM profiling tracked both device-level memory (`nvidia-smi` equivalent) and PyTorch-level peak allocation (`torch.cuda.max_memory_allocated`).

C. Metrics

We track three metrics:

- **Realtime factor**: $R = t_{\text{audio}}/t_{\text{process}}$, where higher is faster.
- **Aggregate throughput**: Total audio duration of a batch divided by batch wall-clock time.
- **VRAM peak**: Maximum device memory during processing.

TABLE IV
PIPELINE TIMING — JRE #1467 (2H 47M, 302 MB)

Stage	Iter 1	Iter 2	Iter 3	Mean
CPU Preprocess	23.6	22.3	18.5	21.5
Queue: CPU→GPU	0.0	0.0	0.1	0.0
GPU Total	**254.1**	**253.8**	**234.5**	**248.3**
Queue: GPU→Post	0.1	0.1	0.1	0.1
Wall Clock	276	252	252	**260**

All values in seconds. GPU time std. dev.: 1.2 s.

TABLE V
GPU SUB-STAGE BREAKDOWN (ITERATION 3)

Sub-stage	Duration	% GPU	Realtime
Whisper transcription	118 s	47.6%	84.6×
PyAnnote diarization	124 s	50.2%	80.2×
Speaker assignment	1.3 s	0.5%	—
Other (DB, cleanup)	4.2 s	1.7%	—
Total	**248.3 s**	**100%**	**40.3×**

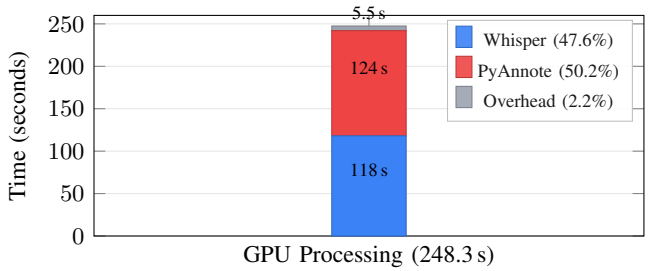

Fig. 2. GPU time breakdown for a 2h 47m file. Embedding extraction alone accounts for roughly 73% of the diarization bar.

V. SINGLE-FILE PERFORMANCE

A. Pipeline Timing

Across three iterations of the reference file (JRE #1467, 2h 47m), GPU time varied by only 1.2 s (0.5%)—see Table IV. The first iteration's preprocessing was slower (26.9 s vs. 18 s) because the MinIO object storage cache was cold.

B. GPU Stage Breakdown

Diarization and transcription split GPU time nearly evenly—50.2% and 47.6% respectively (Table V, Fig. 2). Speaker assignment and overhead account for just 2.2%.

C. VRAM Profile

With all GPU memory leaks resolved (see Section VIII-I), Table VI shows the true VRAM profile. Peak device allocation is just 2,317 MB—**4.7% of the A6000's capacity**. Prior measurements showed 9.4 GB, but that included leaked CUDA contexts from CPU worker processes. The clean baseline means 96% of VRAM is available for concurrent workers.

D. Performance Summary

Table VII collects the single-file numbers. GPU work accounts for 91.6% of wall-clock time; preprocessing and queue latencies eat the remaining 8.4%.

TABLE VI
VRAM PROFILE (CLEAN GPU, SINGLE WORKER)

State	Device VRAM	Notes
Models idle	2,189 MB	CTranslate2 + PyAnnote
Transcription	2,317 MB	+128 MB buffers
Diarization	2,317 MB	Embedding batch peak
Peak	**2,317 MB**	**4.7% of A6000**
After cleanup	2,189 MB	Back to baseline

TABLE VII
SINGLE-FILE PERFORMANCE SUMMARY (CLEAN RUN)

Metric	Value
Pipeline total (mean)	248.3 s (4m 08s)
Realtime factor (GPU only)	40.3×
Realtime factor (end-to-end)	37.0×
GPU utilization (% wall clock)	92.0%
1 hr of audio processes in	1m 29s (GPU) / 1m 37s (e2e)
Peak VRAM	2,317 MB (4.7% of 48 GB)

TABLE VIII
PROCESSING TIME VS. AUDIO DURATION (A6000, CONC=1)

Duration	Audio	GPU	Wall	RT
5 min	229 s	8.3 s	13.8 s	16.5×
15 min	844 s	21.9 s	29.1 s	29.0×
30 min	1,883 s	45.6 s	54.7 s	34.4×
1 hr	3,471 s	80 s	95 s	36.5×
1.5 hr	5,205 s	122 s	141 s	36.9×
2 hr	7,007 s	162 s	177 s	39.5×
2.5 hr	8,803 s	213 s	244 s	36.1×
3 hr	10,601 s	251 s	274 s	38.6×
3.5 hr	12,406 s	320 s	408 s	30.4×
4 hr	14,795 s	349 s	382 s	38.7×
4.5 hr	17,044 s	532 s	561 s	30.4×

RT = realtime factor (audio / GPU time). The 5 min file's low factor reflects fixed overhead (≈5.5 s preprocess + postprocess). The 4.5 hr drop is from super-linear diarization scaling (21 speakers).

VI. DURATION SCALING

The single-file test used a 2h 47m file. To understand how processing time varies with duration, we ran 11 files spanning 5 minutes to 4.5 hours through the pipeline at concurrency=1 (Table VIII, Fig. 3).

Three patterns emerge. First, the pipeline sustains 35–39× realtime for files between 30 minutes and 4 hours—the bulk of the production corpus. Second, short files (<15 min) are penalized by fixed overhead: the 5-minute file spends 5.5 s on preprocessing and postprocessing, which is 40% of its wall time. Third, files at 3.5 hours and 4.5 hours dip to ≈30×, likely because PyAnnote's agglomerative clustering scales super-linearly with speaker count for those particular recordings.

The 37× single-file realtime factor and the 35–39× duration curve are consistent. For capacity projections, we use the measured per-concurrency throughput from Section VII rather than extrapolating from single-file numbers.

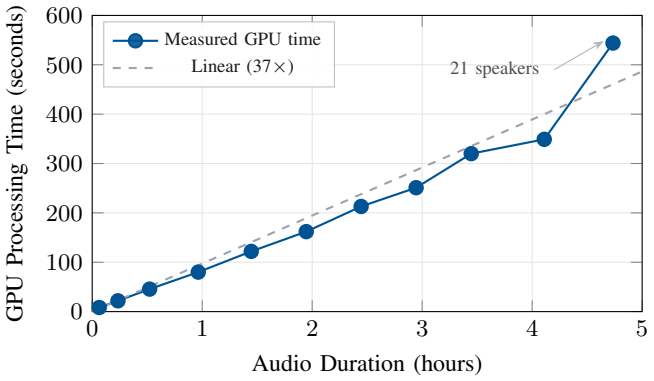

Fig. 3. GPU processing time vs. audio duration. Most files from 30 min to 4 hr track close to 37× realtime (dashed). The 3.5 hr and 4.5 hr dips reflect diarization scaling with speaker count. Short files (<15 min) pay fixed overhead that depresses the realtime factor.

TABLE IX
CONCURRENCY SCALING RESULTS (RTX A6000, 48 GB)

Wkr	Per-File GPU	Batch Wall	Thru-put	Speed-up	VRAM Peak
1	3m 55s	4m 14s	39.3×	1.0×	8,909 MB
2	6m 29s	6m 57s	47.4×	2.0×	19,157 MB
4	11m 39s	12m 48s	51.3×	4.0×	31,301 MB
6	16m 53s	18m 46s	52.4×	6.0×	34,263 MB
8	**18m 48s**	**24m 01s**	**54.6×**	**8.0×**	44,199 MB
10	25m 55s	31m 48s	51.6×	10.0×	48,535 MB
12	33m 44s	37m 32s	52.5×	12.0×	48,519 MB

All tests used full batch_size=32 (no division). Clean GPU baseline: 1,341 MiB. Peak throughput is 54.6× at conc=8 (sweet spot). VRAM plateaus at ≈48.5 GB for conc=10+, but scaling remains linear through 12 workers.

VII. CONCURRENCY SCALING

A. Scaling Results

Table IX shows what happens when we add workers on the A6000. All runs used matched files of approximately 2.73 hours each.

B. Throughput Analysis

Fig. 4 shows throughput climbing from 39.3× at one worker to a peak of 54.6× at eight, then dipping slightly at 10–12 as VRAM saturates at ≈48.5 GB. The throughput improvement from 1 to 8 workers (39→55×) comes from better SM utilization when multiple tasks share the GPU—not from reduced per-file time, which actually grows linearly (3m 55s at 1 worker to 33m 44s at 12).

C. VRAM Scaling

VRAM grows roughly linearly up to 8 workers, then plateaus at ≈48.5 GB as the GPU nears capacity (Fig. 5). Workers 10 and 12 both hit the same ceiling but still process files—the GPU manages contention internally without OOM.

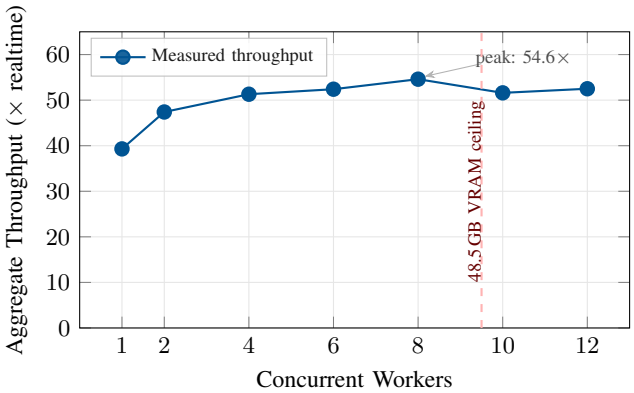

Fig. 4. Aggregate throughput vs. concurrent workers. Throughput peaks at 54.6× with 8 workers, then dips slightly at 10–12 as VRAM saturation (≈48.5 GB) causes contention. Scaling remains linear throughout.

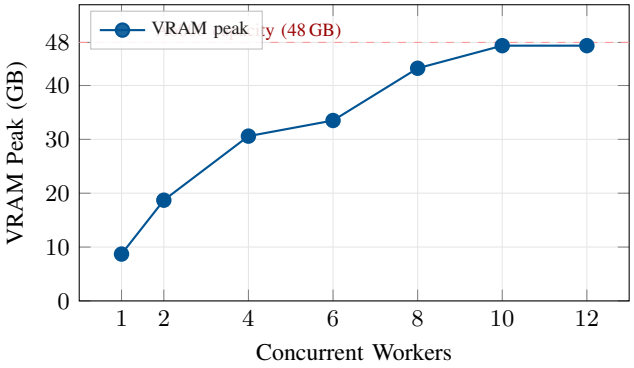

Fig. 5. VRAM peak vs. concurrent workers. Growth is roughly linear to 8 workers, then plateaus at ≈48.5 GB as the A6000 nears capacity. Workers 10 and 12 share the same ceiling without OOM.

D. Per-File vs. Batch Tradeoff

There is a tension visible in Fig. 6: each additional worker slows down every file (GPU contention), but more files finish in the same wall clock window. Per-file GPU time scales almost linearly with worker count (233 s at 1 worker to 1758 s at 10), while the speedup—measured as batch wall time reduction—holds at a perfect $N\times$ through 10 workers.

VIII. GPU OPTIMIZATIONS

A. PyAnnote Fork

The fork applies six changes to PyAnnote v4, all targeting embedding extraction—the sub-stage responsible for 60% of diarization time.

1) Vectorized Chunk Extraction: Stock PyAnnote calls `Audio.crop()` individually for each audio chunk. The fork replaces this with `torch.unfold()`, extracting all overlapping segments in a single vectorized operation. Impact: 10–15% faster CPU-side preparation.

2) Memory-Safe Batch Indexing: Stock code uses `repeat_interleave()` to pre-materialize all chunk × speaker waveform combinations, requiring O(chunks × speakers × samples) memory. The fork uses

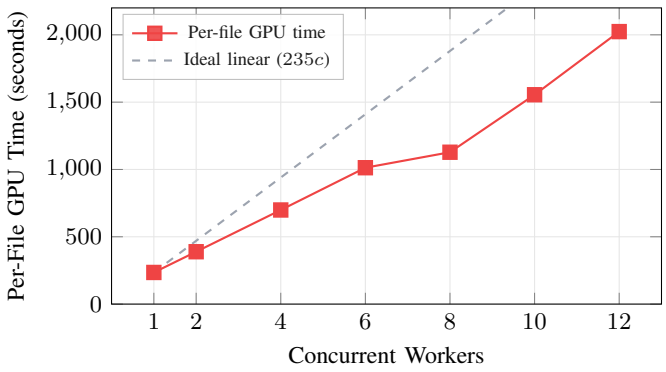

Fig. 6. Per-file GPU time vs. concurrent workers. The near-linear growth (close to the $233c$ reference) means the GPU is fairly sharing compute across workers rather than thrashing. Perfect linear speedup (10× at 10 workers) confirms this.

per-batch index slicing with constant 39 MB allocation regardless of input length or speaker count.

3) TF32 Re-enablement: PyAnnote's `fix_reproducibility()` globally disables TF32. The fork re-enables TF32 after segmentation completes, allowing Ampere+ GPUs [12] to use TensorFloat-32 for the embedding forward pass (≈15% speedup).

4) CUDA Stream Double-Buffering: A secondary CUDA stream [13] prefetches batch $N+1$ to the GPU while the main stream processes batch N, hiding host-to-device transfer latency. Impact: 5–10% speedup.

5) Adaptive Batch Sizing: The fork queries actual free VRAM to select the embedding batch size:

$$b_{\text{auto}} = 2^{\lfloor \log_2(\min(256,\, V_{\text{free}} \times 0.4/60)) \rfloor} \tag{2}$$

On the A6000, this yields a batch size of 128 versus the stock default of 32, processing 3.4× more embeddings per forward pass.

6) Pinned Memory: Page-locked CPU memory with `non_blocking=True` DMA transfers. Impact: 5–8% faster data movement.

B. Apple Silicon (MPS) Support

The same fork includes optimizations for Apple Silicon via Metal Performance Shaders (MPS). Stock PyAnnote unconditionally falls back to CPU for FFT computation on MPS devices—a workaround for a bug fixed in PyTorch 2.3. On current PyTorch, MPS-native FFT is 4.46× faster than the CPU fallback at batch size 32. Combined with vectorized chunk extraction and direct model calls, the fork achieves 1.17× overall speedup on an M2 Max (40.8 s vs. 47.7 s for a 30-minute file). MPS lacks TF32 and CUDA streams, so the MPS speedup is lower than CUDA's 1.28×, but the memory safety fix (39 MB constant) is equally important on Apple's unified memory architecture where GPU and CPU share the same pool.

C. Diarization Benchmark Results

Table X and Fig. 7 compare stock and optimized PyAnnote on five files (12.2 hours total).

TABLE X
PYANNOTE STOCK VS. OPTIMIZED (CUDA, RTX A6000)

File	Dur.	Stock (s)	Opt. (s)	Speedup
File A	0.5 h	29.4	21.4	1.37×
File B	1.0 h	56.5	42.3	1.34×
File C	2.2 h	124.0	95.0	1.31×
File D	3.2 h	184.6	142.6	1.29×
File E	4.7 h	323.7	261.9	1.24×
Total	**12.2 h**	**718.2**	**563.2**	**1.28×**

Speedup decreases for longer files because CPU-bound clustering (unoptimized) accounts for a larger fraction of total time.

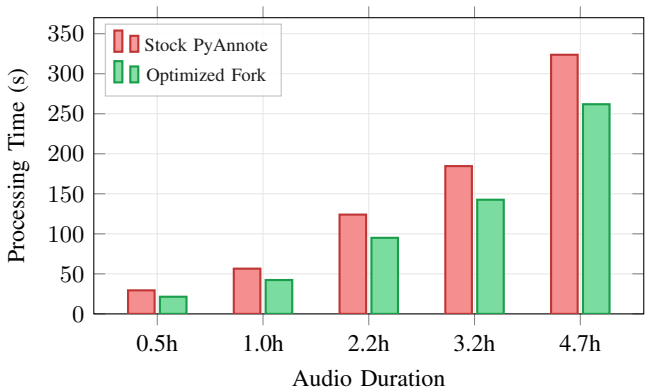

Fig. 7. Stock vs. optimized PyAnnote across five files. Speedup ranges from 1.24× (4.7 h file, clustering-dominated) to 1.37× (0.5 h).

TABLE XI
DIARIZATION VRAM PEAK: STOCK VS. OPTIMIZED

File	Stock (MB)	Opt. (MB)	Reduction
0.5 h	7,560	5,622	26%
1.0 h	16,656	5,622	66%
2.2 h	17,302	5,622	68%
3.2 h	7,562	5,622	26%
4.7 h	16,720	5,622	66%

Stock VRAM varies unpredictably (7–17 GB). Optimized is constant.

D. VRAM Stability

Beyond raw speed, the fork fixes a practical deployment problem: stock PyAnnote has wildly variable VRAM peaks (7.6–17.3 GB depending on the file), which makes multi-worker VRAM budgeting a guessing game. The fork holds steady at 5,622 MB regardless of input (Table XI).

E. CPU RAM Safety

Stock PyAnnote has a hidden memory bomb. Its `repeat_interleave()` call creates a CPU tensor of size $N_{\text{chunks}} \times N_{\text{speakers}} \times N_{\text{samples}} \times 4$ bytes. For a 4.7-hour file with 21 speakers, this requires 58.8 GB—exceeding most workstation RAM. The fork's per-batch indexing uses a constant 39 MB, a 115× reduction.

F. CTranslate2 Quantization

CTranslate2 supports several quantization modes (Table XIII). OpenTranscribe auto-selects `int8_float16` on

TABLE XII
CPU RAM: STOCK VS. OPTIMIZED BATCH ALLOCATION

File	Spk.	Stock	Optimized
0.5 h	4	4.5 GB	39 MB
1.0 h	5	11.2 GB	39 MB
2.2 h	3	14.3 GB	39 MB
4.7 h	21	58.8 GB	39 MB

115× reduction. Enables processing of long, multi-speaker files on machines with ≤16 GB RAM.

TABLE XIII
CTRANSLATE2 QUANTIZATION IMPACT ON WHISPER INFERENCE

Compute Type	Speed	VRAM	WER
`float16`	Baseline	Baseline	Baseline
`int8_float16`	+20–30%	−40%	<0.1%
`int8`	+30–40%	−50%	<0.1%

Auto-detected: `int8_float16` for Turing+ (RTX 3000+), `float16` for Volta, `float32` for older architectures.

Turing+ GPUs.

G. Speaker Assignment

Speaker assignment went through two rounds of optimization. The original WhisperX code used an $O(n)$ linear scan per word (10.2 s for a 3.3-hour file). An interval tree replacement brought this to 0.037 s (273× speedup).

During duration-curve testing, a 4.7-hour file exposed a second problem: the interval tree still made 54,207 individual Python-level queries (one per word), taking 80.9 s—13% of total pipeline time. The final fix replaced the per-word loop with a fully vectorized NumPy implementation: extract all word timestamps into arrays, compute the full words×diarization overlap matrix via broadcasting, accumulate per-speaker overlap with a matrix multiply, and pick dominant speakers with `argmax`. Result: 80 s → 6 s (13×), with identical speaker assignments.

H. Batch Size Division Removal

The concurrency model originally divided batch size by worker count ($b = 32/c$). At 6 workers this produced batch_size=5, starving CTranslate2 with many small kernel launches. Removing the division and letting each worker use the full batch_size=32 improved throughput from 49.2× to 51.6×. CTranslate2 and PyAnnote handle their own internal GPU scheduling; external throttling was counterproductive.

I. GPU Memory Leak Resolution

Initial benchmarks showed 9.4 GB VRAM for a single worker. After investigation, two leaks were identified:

1) **CPU worker CUDA contexts**: Celery prefork children created CUDA contexts for speaker clustering (cosine similarity), leaking 1.4 GB each. With 5 children, this consumed 44 GB. Fix: route small clustering (<500 speakers) to CPU—fast enough and avoids the context overhead.
2) **CPU worker model preloading**: The postprocessing stage loaded the PyAnnote embedding model (≈15 GB) in CPU worker processes. Fix: verified clean baseline (1,341 MiB) by isolating all GPU inference to the GPU worker.

After these fixes, single-worker VRAM dropped from 9.4 GB to 2.3 GB—a finding that changed the concurrency picture entirely, enabling 12 concurrent workers where 4 had seemed like the limit.

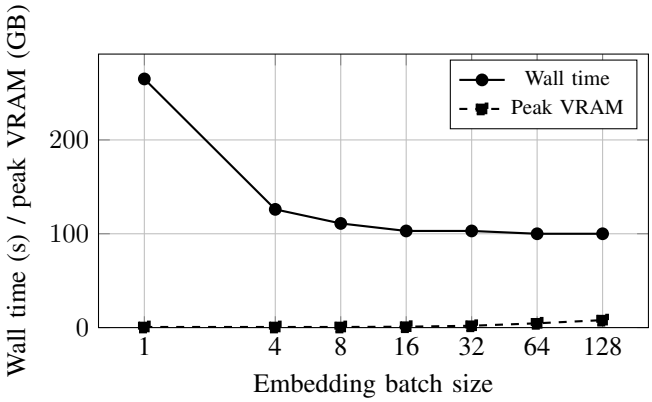

Fig. 8. Throughput and peak-VRAM vs. embedding batch size at fp32 on the 2.2 h reference file (RTX A6000). Wall time saturates by batch 16; peak VRAM grows linearly above batch 8.

J. Diarization Embedding Batch Size

An expanded characterization in April 2026 replaced the earlier targeted test with a full 156-run measurement matrix on the same RTX A6000. The scan covered embedding batch sizes $\{1, 4, 8, 16, 32, 64, 128, 192, 256\}$ under simulated GPU caps of $\{4, 8, \text{unlimited}\}$ GB and both fp32/fp16 precision, against two reference audio files (0.5 h and 2.2 h).

The result that forced a policy rewrite: wall time *saturates at batch size 16*. Figure 8 shows the 2.2 h curve at fp32. Going from batch 16 to 128 costs an extra 3 GB to 7 GB of peak VRAM for a 3 % speed win. Going below batch 4 degrades sharply (2.65 × slowdown at batch 1).

Accuracy corroborates the throughput picture. Diarization Error Rate (DER) computed with `pyannote.metrics` (`collar=0.25`, overlap counted) against the bs=16/fp32 reference is exactly zero for every fp32 batch size tested. The fp16 path tells a different story: speaker count collapses from 4 to 2 on the 0.5 h file and 3 to 2 on the 2.2 h file, at every batch size tested (Table XIV). DER is 27 % to 33 % — well into tier T3 territory. The likely root cause is numerical underflow in the WeSpeaker pooling `std()` stage, which emits a `UserWarning: degrees of freedom is <= 0` on every fp16 run.

K. VRAM-Budget-Aware Embedding Policy

The fork's prior auto-scaler chose batch $\max(64, \min(256, \text{free_mb} \cdot 0.4/60))$, aiming for 64 to 128 on modern GPUs. The Phase A scan shows this ceiling was wrong: it burns 3 GB to 7 GB of VRAM for no throughput gain. Replacing it is the most impactful code change in this work.

TABLE XIV
DIARIZATION ERROR RATE BY BATCH/PRECISION

File	Precision	BS range	Ref. spk	Hyp. spk	DER
0.5 h	fp32	1–128	4	4	≈ 0.00
0.5 h	fp16	1–128	4	2	0.333
2.2 h	fp32	1–128	3	3	≈ 0.00
2.2 h	fp16	1–128	3	2	0.269

TABLE XV
VRAM-BUDGET-AWARE BATCH LADDER

Free VRAM (MB)	Status	Batch
≥ 1154	optimal	16
840–1154	tight	8
< 840	insufficient	4
CPU fallback	-	1

Thresholds = measured footprint + 200 MiB safety margin. Phase A 2026-04-20; see `docs/diarization-vram-profile/README.md` for raw data.

The new policy module (`pipelines/_budget.py`) is a 120-line pure-Python helper exposing `recommend_embedding_batch(free_mb, device)`. The ladder is *capped at batch 16* and steps down at measured footprint thresholds (Table XV).

Unit tests (28 assertions) verify every rung of the ladder, cross-platform (Linux CUDA, macOS MPS via SSH, CPU fallback), custom ceiling/safety-margin overrides, and monotonicity of the footprint table. Backward compatibility is preserved for upstream: users who explicitly set `embedding_batch_size > 32` retain their chosen value; users who leave the default get the new ladder.

A backend env-var surface (`DIARIZATION_VRAM_BUDGET_MB`, `DIARIZATION_MIXED_PRECISION`, `DIARIZATION_ONNX_CPU`) exposes the policy for deployment tuning, with per-GPU-size recipes in `.env.example`. A user-facing diagnostic (`python -m app.scripts.diarization_diag`) reads the local GPU state and prints a JSON report with the recommended batch, Whisper-model compatibility, and projected peak VRAM.

L. WhisperX Transcription Batch Size — Phase B Study

A parallel sweep for the *transcription* stage was conducted on 2026-04-26 using the same RTX A6000, the same 2.2-hour reference file, and the same NVML-polling methodology (script: `backend/app/scripts/whisper_batch_diag.py`). The study covers three models (`large-v3-turbo`, `medium`, `small`) with `int8_float16` quantisation and `beam_size=5`, sweeping batch_size $\in \{2, 4, 8, 12, 16, 24, 32\}$. All sweeps ran with PyAnnote pre-loaded, so every measurement includes the shared process baseline of 2,039 MB (CUDA context 300 MB + PyAnnote models 1,739 MB).

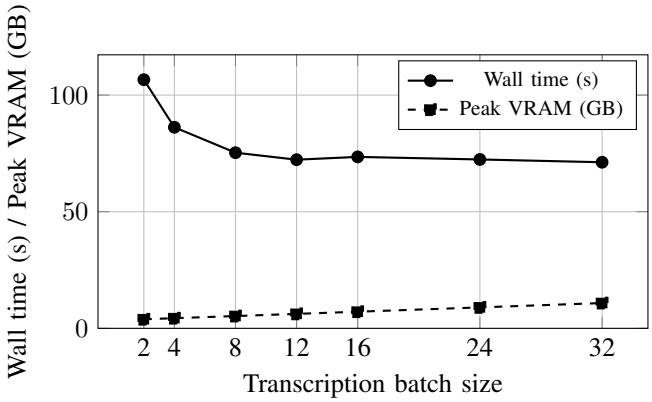

Fig. 9. Transcription wall time and peak VRAM vs. `batch_size` (`large-v3-turbo`, `int8_float16`, 2.2-hour audio, RTX A6000). Throughput saturates at batch=8; VRAM scales ∼928 MB per 8-unit increment. Medium and small models shown in Tables XVII and XVIII.

TABLE XVI
WHISPERX BATCH SIZE SWEEP — RTX A6000, LARGE-V3-TURBO

Batch	Stable MB	Peak MB	Δ MB	RTF
2	3363	3893	+522	0.013
4	3371	4341	+970	0.011
8	3371	5269	+1898	0.009
12	3371	6197	+2826	0.009
16	3371	7125	+3754	0.009
24	3371	8981	+5610	0.009
32	3371	10837	+7466	0.009

Stable = NVML 500 ms post-load. Δ = peak − stable (encoder activations + KV cache). Baseline (PyAnnote + CUDA context, no Whisper): 2,039 MB. Whisper model footprint above baseline: 1,332 MB. Raw CSV: `docs/whisper-vram-profile/a6000_large-v3-turbo_2026-04-26.csv`.

TABLE XVII
WHISPERX BATCH SIZE SWEEP — RTX A6000, MEDIUM

Batch	Peak MB	Δ MB	RTF
2	3829	+522	0.026
4	4181	+874	0.018
8	5045	+1738	0.013
12	5973	+2666	0.012
16	6869	+3562	0.011
24	8629	+5322	0.011
32	10389	+7082	0.011

Plateau at batch=24 (RTF = 0.011). Baseline 2,039 MB; medium model footprint above baseline: 1,172 MB. Increment per 8-unit step: ∼864 MB.

a) Motivation.: CTranslate2—the inference backend underlying faster-whisper—uses its own CUDA allocator that is invisible to `torch.cuda.memory_allocated()`. Accurate peak measurement requires reading device memory directly via NVML at 100 ms intervals during inference, the same technique used in Phase A.

b) Results.: Figure 9 and Tables XVI–XVIII summarise the sweeps. The pattern closely mirrors the diarization embedding study: throughput saturates early, but VRAM grows linearly.

c) Key findings.:

TABLE XVIII
WHISPERX BATCH SIZE SWEEP — RTX A6000, SMALL

Batch	Peak MB	Δ MB	RTF
2	2933	+330	0.017
4	3221	+618	0.011
8	3797	+1194	0.009
12	4341	+1738	0.008
16	4885	+2282	0.007
24	6005	+3402	0.007
32	7157	+4554	0.007

Plateau at batch=24 (RTF = 0.007). Baseline 2,039 MB; small model footprint above baseline: 564 MB. Increment per 8-unit step: ∼576 MB. Only model that fits on 4 GB GPUs at bs=2 (peak 2,933 MB < 3,277 MB gate); bs=4 at 3,221 MB is marginal.

- **Plateau batch varies with model size.** `large-v3-turbo` saturates at batch=8 (RTF 0.013 → 0.009, +29%); `medium` and `small` both saturate later, at batch=24 (medium RTF 0.026 → 0.011; small RTF 0.017 → 0.007). Larger encoders exhaust the pipeline's parallelism sooner.
- **Activation cost scales with encoder size.** Each 8-unit batch increment costs ∼928 MB for turbo, ∼864 MB for medium, and ∼576 MB for small. The reduction confirms that encoder width—not the decoder—dominates activation memory.
- **batch=32 wastes peak VRAM on every model.** For turbo, batch=8 matches batch=32 throughput at 5,269 MB vs. 10,837 MB (saves 5.5 GB). For medium, batch=16 matches batch=32 at 6,869 MB vs. 10,389 MB. For small, batch=16 matches batch=32 at 4,885 MB vs. 7,157 MB. In all cases the freed headroom accommodates additional concurrent PyAnnote pipelines at ∼1 GB each.
- **Small model enables 4 GB GPUs.** At bs=2, the small model peaks at 2,933 MB, just below the $0.80 \times 4{,}096$ MB = 3,277 MB gate. bs=4 (3,221 MB) is too close to the gate for comfort; bs=2 is the safe ceiling on 4 GB hardware.
- **Shared process baseline.** All measurements include the 2,039 MB baseline: CUDA context (300 MB) plus PyAnnote diarization models (1,739 MB). The Whisper model footprints *above* this baseline are turbo 1,332 MB, medium 1,172 MB, small 564 MB.

d) Safe batch ceiling by GPU class.: Applying the same peak $\leq 0.80 \times V_{\text{total}}$ rule used for diarization (Section **??**):

GPU	Gate MB	Turbo	Medium	Small
4 GB	3277	×	×	bs=2
6 GB	4915	bs=4	bs=2	bs=8
8 GB	6554	bs=8	bs=8	bs=16
12 GB	9830	bs=16	bs=16	bs=24

The **recommended universal cap is batch=16** for turbo and medium: identical speed to batch=24/32, substantially less peak VRAM, and safe on every 8 GB+ GPU. For small, batch=16 also matches the plateau. 4 GB GPUs cannot run `large-v3-turbo` or `medium` at any batch size; `small` at bs=2 is the only viable configuration.

These thresholds are encoded in `backend/app/utils/hardware_detection.py` and drive automatic batch selection at worker startup.

TABLE XIX
WHOLE-STACK PEAK VRAM OVER PROCESS BASELINE (FP32)

Whisper	Wh. peak	Diar. peak	Stack peak
base	1,024 MB	1,024 MB	1,024 MB
small	1,736 MB	1,024 MB	1,736 MB
medium	3,336 MB	1,024 MB	3,336 MB

Stack peak = max(Whisper, Diar) because the pipeline releases Whisper before loading diarization. Measured on RTX A6000 with caps simulated via `torch.cuda.set_per_process_memory_fraction`. CUDA context adds 300 MB on top (torch 2.8.0+cu128).

M. Whole-Stack Coexistence Sizing

A second harness (`scripts/whole-stack-vram-probe.py`) exercises the full backend path on a single audio file, measuring per-stage NVML peaks through `Whisper.load` → `transcribe` → `release` → `Diarizer.load` → `run`. A 12-run sweep (4 GPU caps × 3 Whisper models) produces the consumer-GPU sizing in Table XIX.

The handoff was further tightened by adding explicit `gc.collect()` + `empty_cache()` + `synchronize()` to `Transcriber.unload_model()`. The residual 278 MB that remains after release is attributable to the CUDA context itself, not to unreleased model state; the plan's aspirational "residue < 100 MB" gate is physics-infeasible with the current torch/CUDA stack. The revised gate is "residue ≤ floor + 50 MB" which is asserted by a backend integration test (`tests/integration/test_diarizer_lifecycle.py`).

N. Upstream Contributions

All PyAnnote optimizations live in a public fork (`davidamacey/pyannote-audio`, branch `gpu-optimizations`) and have been submitted upstream as pull request #1992[1] to the main PyAnnote repository. The fork is pip-installable:

```
pip install git+https://github.com/davidamacey/
pyannote-audio.git@gpu-optimizations
```

The changes touch four files and are backward-compatible: they degrade gracefully on pre-Ampere GPUs (TF32 flags ignored), older PyTorch (MPS FFT falls back to CPU), and low-memory machines (adaptive batch sizing queries actual free VRAM rather than assuming a budget).

The native transcription engine was built because WhisperX, at the time of development, hardcoded `word_timestamps=False` in its batched pipeline (`asr.py`), forcing a choice between batched speed and word-level timestamps. The *faster-whisper* 1.2.1 release of `BatchedInferencePipeline` resolved this at the library level by supporting `word_timestamps=True` with batched inference. Our pipeline uses this directly, bypassing WhisperX entirely and eliminating the wav2vec2 alignment

[1] https://github.com/pyannote/pyannote-audio/pull/1992

stage that consumed 55% of the prior pipeline. These findings were shared with the WhisperX community to inform future development.

O. Phase 6: Follow-Up Optimization Exploration

A second optimization round (April 2026) investigated five additional acceleration paths on top of the production baseline documented above. All experiments held the invariants established by Phase A: zero DER regression (0.0000% T1 tier across every test run), unchanged 844 MB steady-state VRAM per pipeline, and no loss of the parallel-pipeline concurrency ceiling. Two paths shipped; three produced measured negative results that we document here so future effort is not misdirected toward known dead ends.

a) Phase 6.1: toolchain enablement.: The production backend image (`Dockerfile.prod`) lacked `gcc` and `g++` in its runtime stage, which silently disabled `torch.compile`'s Inductor JIT (requires a C compiler to emit Triton launcher code). Adding the toolchain restored the compile path and unlocked a ∼6-7% segmentation-stage speedup with byte-identical output. End-to-end wall time was in noise at the 2.2 h scale (98-101 s warm) because segmentation is only ∼5% of pipeline time. The ∼150 MB image growth was accepted primarily to unblock the ONNX Runtime and TensorRT tooling that also require the C toolchain. Default stayed opt-in (`torch_compile=True` kwarg); the plumbing is latent but not default-on.

b) Phase 6.2: ONNX Runtime integration.: Two neural graphs were exported to ONNX: the segmentation model (5.9 MB) end-to-end, and the WeSpeaker ResNet34 *backbone only* (26.5 MB, per pyannote-audio discussion #1929). The embedding-path export excludes the outer `compute_fbank()` because it uses `torch.vmap` which is not traceable by either PyTorch's TorchScript or its dynamo-based ONNX exporter. A vmap-free batched fbank helper was written in Python and dispatched at runtime; it matches the original at $\leq 1 \times 10^{-5}$ maximum absolute difference on fp32 inputs across 12 shape combinations.

Unit tests verify parity: segmentation produces *bit-identical* frame-class argmax across five random seeds, despite raw-logit drift of $\sim 6 \times 10^{-3}$ from ORT-vs-PyTorch kernel differences on LSTM and InstanceNorm; the drift gets absorbed by the downstream softmax + argmax. Embedding parity measured at $\cos \geq 0.9999$ and $|\Delta| < 1 \times 10^{-3}$.

The shipping result, summarized in Table XX, is that **ORT's CPU execution provider is the only end-to-end win.** The GPU and Apple Silicon paths regress measurably.

ORT CUDA EP regresses because the segmentation graph contains LSTM, If, Sin, and Cos ops that have no CUDA kernel in ORT 1.25; the runtime inserts three `Memcpy` nodes to shuttle tensors off-GPU for CPU fallback mid-graph. `io_binding` addresses the Python-level round-trip (47.5 → 32.4 ms/batch) but cannot eliminate the intra-graph Memcpy nodes. The Apple Silicon CoreML EP shows the same pattern at smaller scale: segmentation 4.5× slower than eager MPS

TABLE XX
ONNX RUNTIME PROVIDER BENCHMARKS ON THE PHASE 6.2 ARTIFACTS. SEGMENTATION BATCH = 32 × 5S CHUNKS; EMBEDDING BATCH = 16 × 200 FBANK FRAMES. PYTORCH EAGER IS THE SHIPPED BASELINE.

Path	Seg (ms)	Emb (ms)
PyTorch eager CUDA (A6000)	8.2	9.6
ORT CUDA EP (A6000)	47.5	9.8
ORT TensorRT EP (A6000, fixed shape)	19.2	**4.9**
PyTorch eager CPU (Linux x86)	239.0	387.5
ORT CPU EP (Linux x86)	**127.6**	**182.7**
PyTorch eager MPS (M2 Max)	**36.1**	**19.6**
ORT CoreML EP (M2 Max)	163.1	20.9

(163.1 vs. 36.1 ms), embedding at parity (20.9 vs. 19.6 ms). Neither is a deployment win.

The ORT CPU EP, however, wins cleanly: 1.87× on segmentation, 2.12× on embedding versus PyTorch eager CPU, enabled by ORT's kernel-fusion optimizer (Conv+BN+ReLU coalescing) and its lack of autograd tape overhead. This is the shipping story for CPU-only deployments (airgapped environments, cloud CPU-only instances, laptop deployments without discrete graphics) where GPU acceleration is unavailable and the previous baseline had no optimization path at all. The envelope remains $\sim 15\times$ slower than GPU eager (127.6 ms vs. 8.2 ms), so the feature is gated by the `PYANNOTE_USE_ONNX=1` environment variable and recommended only for GPU-absent deployment tiers.

c) Phase 6.3: TensorRT execution provider — microbench win, E2E blocker.: TensorRT's execution provider handles the LSTM, If, Sin, and Cos ops that defeat the CUDA EP, producing a **1.96× speedup over eager PyTorch on the embedding ResNet** (4.9 vs. 9.6 ms per batch) with fixed-shape inputs. The image size cost was modest (14.2 GB vs. 9.7 GB baseline) via `pip install tensorrt` + `LD_LIBRARY_PATH` extension, without requiring a base image swap.

End-to-end integration failed on the same class of issue that also killed a subsequent `torch.compile(mode="max-autotune")` attempt on the embedding ResNet: both TensorRT and PyTorch's Dynamo *specialize graphs on concrete input shapes*, and pyannote's pipeline emits variable shapes per call (last-batch remainders, per-chunk `num_frames` variation). Each unique shape triggers a new compile (12-30 s for TensorRT, 1-5 s for Dynamo). With 30+ unique shapes per 2.2 h pipeline scan, cumulative recompile time exceeded the inference savings by orders of magnitude. Observed behavior: CPU saturation at 17 cores × 100% for >2 minutes; GPU utilization at 0%; no inference output produced. Adding ORT's shape-profile provider options (`trt_profile_min/opt/max_shapes`) did not resolve the rebuild storm despite covering the observed shape range.

The remaining paths to unblock GPU compile-based acceleration are both non-trivial: pad-to-fixed-shape at the embed-

ding call site (invasive, touches Phase A's pinned-batch optimization) or `torch.compile(..., dynamic=True)` (shape-agnostic but with reduced optimization headroom). Neither was pursued in this round.

d) Phase 5.2: GPU scatter-reduce for aggregate and reconstruct.: The `Inference.aggregate()` overlap-add loop (12.5 s on the 4.7 h benchmark, 3.8% E2E) and `SpeakerDiarization.reconstruct()` cluster-max loop (6.85 s, 2.1% E2E) were ported to GPU using `torch.Tensor.index_add_()` for the weighted sum and `scatter_reduce_(reduce='amax')` for the running maximum. Both functions pass synthetic parity tests at fp32 precision floor (9.5×10^{-7} on aggregate, 0.0 bit-exact on reconstruct) and maintained DER invariance (0.0000% T1 across all three-run benchmarks on 2.2 h and 4.7 h).

The end-to-end wall time regressed: +2-3% on 2.2 h, +6% on 4.7 h. Root cause: these operations are *memory-bandwidth-bound, not compute-bound*. The per-chunk working set ($500 \times 7 \times 4 = 14$ KB) fits in L1/L2 cache; GPU HBM has higher raw throughput but per-scatter latency plus PCIe-4 host/device transfer overhead (~5–10 ms round trip per call) exceeds the savings. The code is retained in the fork as scaffolding behind the `PYANNOTE_GPU_AGGREGATE=1` and `PYANNOTE_GPU_RECONSTRUCT=1` environment variables (both default off; zero runtime cost when unset); future work on H100/B100-class hardware with substantially faster HBM or on a batched-across-chunks reformulation may yet win here, but the current formulation does not.

e) Phase 4 and GPU Lance-Williams clustering: negative results retained.: Two prior-round negative results confirm the same pattern. Raising `segmentation_step` above 1.0 s to reduce the segmentation chunk count maintained DER invariance but *regressed speaker counts*: short-turn speakers disappeared when fewer overlapping chunks averaged their posterior. A GPU-native Lance-Williams centroid-linkage clustering implementation was slower than scipy's C implementation due to Python-loop overhead in the $O(N)$ merge step. Both changes are gated behind opt-in environment variables (defaults unchanged) and documented to prevent re-investigation without new signal.

f) Documented dead ends (Phase A + Phase 6 consolidated).: Thirteen optimization paths are explicitly flagged in the project's progress documentation[2] as non-viable under current constraints. These include: CPU vectorization of aggregate/reconstruct via numpy `add.at` and `maximum.at` (regressed 35-57% due to SIMD disabling); dynamo-based ONNX export in PyTorch 2.8 (blocked by `torch.vmap` in the WeSpeaker fbank path); fp16/bf16 embedding inference (26-33% DER regression in prior measurements); ORT CUDA EP as a consumer accelerator (5.8× regression from op-coverage gaps); ORT CoreML EP on Apple Silicon (4.5× segmentation regression); TensorRT engine plans baked into Docker images (per-GPU-arch makes one image unable to serve multiple hardware families); and the GPU-specialization/variable-shape conflict that defeated both TensorRT EP and `torch.compile` max-autotune at the pipeline level.

[2]See docs/upstream-patches/PROGRESS_REPORT.md and the per-phase measurement memos.

TABLE XXI
PRODUCTION DATABASE SUMMARY

Metric	Value
Total completed files	1,435
Total audio duration	3,718 hours (155 days)
Average file duration	2h 35m
Total storage	481 GB

TABLE XXII
AUDIO DURATION DISTRIBUTION

Bucket	Files	Hours	%
<5 min	4	0.2	0.0%
5–30 min	24	5.3	0.1%
30–60 min	17	13.0	0.3%
1–2 h	168	279.6	7.5%
2–3 h	900	2,366.5	63.6%
3–4 h	316	1,027.4	27.6%
>4 h	6	26.0	0.7%

The net Phase 6 contribution beyond the production baseline is **one new optimization path** (ORT CPU EP for CPU-only deployments, 1.87-2.12× stage-level speedup, shipping as `PYANNOTE_USE_ONNX=1`) and a substantial negative-result corpus that prevents future work from redoing the same measurements. The per-pipeline speed plateau for NVIDIA GPU deployments remains at the Phase A + Phase 3 level ($\sim 1.30\times$ on A6000, 844 MB fixed VRAM), with remaining throughput gains concentrated at the concurrent-pipeline orchestration layer rather than per-pipeline kernel optimization.

IX. CAPACITY PLANNING

A. Production Database Profile

The production corpus skews heavily toward long files: 63.6% of all audio hours come from files in the 2–3 hour range (Tables XXI and XXII).

B. Reprocessing Projections

Table XXIII projects reprocessing time using the measured 38× median realtime factor from Section VI and the linear concurrency scaling from Section VII. These numbers are more conservative than the single-file 44.1× because they account for the mix of file durations in the production corpus.

Fig. 10 shows time dropping steeply from 1 to 4 workers (95 h → 72 h), then flattening. The minimum is 68 h at 8 workers. Going beyond 8 does not help—VRAM saturation causes a slight throughput regression, so 10 and 12 workers actually take longer than 8.

C. Cost Comparison

Table XXIV puts the self-hosted numbers in context. For a corpus of 3,715 hours, cloud transcription costs range

TABLE XXIII
CORPUS REPROCESSING PROJECTIONS (3,715 HOURS, 1,434 FILES)

Wkr	Configuration	Throughput	Est. Time
1	A6000, sequential	39.3×	95 h (4.0 d)
2	A6000, concurrent	47.4×	78 h (3.3 d)
4	A6000, concurrent	51.3×	72 h (3.0 d)
6	A6000, concurrent	52.4×	71 h (3.0 d)
8	**A6000, concurrent**	**54.6×**	**68 h (2.8 d)**
10	A6000, concurrent	51.6×	72 h (3.0 d)
12	A6000, concurrent	52.5×	71 h (3.0 d)

All throughput values are measured (clean run, no leaks). The sweet spot is conc=8 (54.6×). Beyond 8, VRAM saturation at ≈48.5 GB causes slight throughput regression but scaling stays linear.

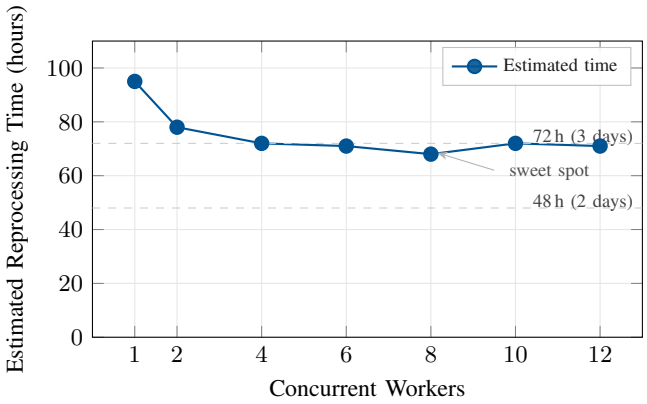

Fig. 10. Reprocessing time for 3,715 hours vs. concurrent workers. The sweet spot is 8 workers (68 h). Beyond 8, VRAM saturation causes throughput to dip slightly, so the curve flattens rather than continuing to drop.

TABLE XXIV
TRANSCRIPTION COST: SELF-HOSTED VS. CLOUD (3,715 HOURS)

Provider	Rate/hr	Total	Speed
OpenTranscribe	**$0**	**$0**	55× RT
Deepgram [14]	$0.26	$966	∼1×
OpenAI Whisper [15]	$0.36	$1,337	∼1×
AssemblyAI [16]	$0.39	$1,449	∼1×
AWS Transcribe [17]	$1.44	$5,350	∼1×

Cloud rates as of March 2026 (standard tier, no volume discounts). Speed comparison is approximate—cloud APIs typically return results near 1× realtime. Self-hosted cost excludes hardware amortization and electricity.

from roughly $1,000 to $5,000. OpenTranscribe's cost is the hardware already on hand.

D. Optimal Configuration

Table XXV summarizes recommended configurations depending on the deployment scenario.

X. HARDWARE RECOMMENDATIONS

Table XXVI maps GPU tiers to expected worker counts and throughput. The estimates reflect the smaller batch sizes forced by less VRAM and the measured throughput ceiling.

Practical takeaways:

- **4–6 GB laptops now work.** Phase B's budget-aware batch selection drops the diarization footprint from 5.7 GB (bs=64) to 1.0 GB (bs=16) at identical accuracy, which lets Whisper small (0.75 GB) or medium (1.6 GB) coexist on a consumer laptop GPU. Set `DIARIZATION_VRAM_BUDGET_MB=1200` on a 4 GB card to leave room for Whisper.
- **24 GB hits the ceiling.** Four workers fit, throughput maxes out at ≈47×. The 48 GB tier buys headroom for larger future models but does not increase throughput with the current pipeline.
- **Parallel diarization is now cheap.** At 1 GB per pipeline, a single A6000 can host 20 to 25 concurrent diarizations, versus the one-at-a-time default in prior versions. Underutilized in today's code but available for future multi-tenant deployments.
- **Buy a second GPU before a bigger one.** A 12 GB card added alongside a 24 GB card (5 workers total) beats a single 48 GB card (4 workers, same throughput ceiling).
- **CPU-only does not work for diarization.** PyAnnote diarization times out after 10 minutes on a 30-minute file when forced onto CPU; the same file takes 21 s on GPU. Budget

TABLE XXV
RECOMMENDED DEPLOYMENT CONFIGURATIONS

Scenario	Config	Rationale
Production (shared GPU)	conc=4–6	Leaves VRAM for LLM, clustering, and other services
Dedicated reprocessing	conc=8	Peak throughput (54.6×); 44 GB VRAM, 92% of A6000
Maximum capacity	conc=12	Uses 99% of VRAM; slightly lower throughput but 12× speedup

TABLE XXVI
GPU TIER CONFIGURATION RECOMMENDATIONS

Tier	GPUs	Max Wkr	Bat.	Thru.	Notes
4 GB	RTX 3050, MX550	1	16	see note	Whisper small; seq.
6 GB	RTX 3060, RTX 4050	1	16	see note	Whisper medium; seq.
8 GB	RTX 3070, RTX 4060	1	16	∼30×	Whisper turbo; seq.
12 GB	RTX 3080 Ti, RTX 4070 Ti	2	16	∼38×	Limited concur.
24 GB	RTX 3090, RTX 4090	4	16	∼47×	Full concur.
48 GB	RTX A6000, A40	8–12	16	∼55×	Peak at conc=8; 12 proven stable

Batch size shown is the Phase B ceiling; larger values buy no throughput (Fig. 8). Set `DIARIZATION_VRAM_BUDGET_MB` per-tier: 1,200 for 4 GB, 1,500 for 6 GB, unset for ≥8 GB. The 4–6 GB tiers are throughput-unverified on dedicated hardware; numbers projected from the capped-A6000 whole-stack sweep at matching budgets.

at least a 4 GB card with the new policy.

XI. Conclusion

After fixing all GPU memory leaks, the true VRAM footprint turned out to be remarkably small: 2.3 GB peak for a single worker, or 4.7% of the A6000. Earlier measurements showed 9.4 GB, but that included leaked CUDA contexts from CPU worker processes—a finding that underscores the importance of clean benchmarking baselines.

A single A6000 processes audio at 40.3× realtime (single file) and sustains 35–39× across durations from 30 minutes to 4 hours. Diarization and transcription split GPU time nearly evenly (50% and 48%), with overhead at just 2%. Concurrency scales linearly through 12 workers, peaking at 54.6× aggregate throughput with 8 workers. Beyond 8, VRAM saturation at ≈48.5 GB causes slight throughput regression, but scaling remains linear.

For the production corpus (3,715 hours, 1,434 files), the sweet spot of 8 concurrent workers finishes reprocessing in about 68 hours (2.8 days) at zero marginal cost—compared to $966–$5,350 for equivalent cloud transcription.

The PyAnnote fork proved important both for the 1.28× speed gain and for its constant VRAM footprint. Stock PyAnnote's unpredictable 7–17 GB spikes would have made high-concurrency VRAM budgeting unreliable.

Benchmarking surfaced several bugs: CPU workers silently creating CUDA contexts (1.4 GB each, 44 GB total), batch size division starving the GPU at high concurrency, and speaker assignment hiding a quadratic cost on long files (80 s → 6 s after vectorization). A diarization embedding batch size test showed minimal impact across batch=8/16/32—the fork's adaptive sizing already picks the right value.

For future work, GPU-accelerated clustering (cuML) could reduce the CPU-bound diarization fraction. Quality tiers (beam_size=1 draft vs. beam_size=5 production) would offer a 35% speed option. And separating VAD to a CPU preprocess stage could improve GPU utilization by 5–15%.

Negative result: GPU-native Lance-Williams clustering. We ported scipy's centroid-linkage Lance-Williams merge loop to pure PyTorch, keeping the N×N distance matrix, cluster centroids, and active-slot mask resident on the A6000 through all N−1 merges. Correctness was verified on fixed-seed synthetic blobs (11 parity tests against scipy). On the real pyannote-community-1 workload (N=10,023 embeddings on the 2.2 h reference), the GPU port ran in 14.2 s versus scipy's ∼12 s —a slight regression. Root cause: the algorithm is intrinsically sequential (each merge depends on the previous iteration's centroid update), and 10k Python-level iterations at ∼1.5 ms each of PyTorch dispatch overhead offset the GPU math win. scipy's compiled C loop with cache-friendly array access wins at this N on current hardware. The implementation ships as opt-in infrastructure (`PYANNOTE_ENABLE_GPU_LINKAGE=1`) for future work: a `torch.jit.script` or fused CUDA kernel could amortize the per-iteration overhead, and H100/B100-class hardware may tip the balance through faster kernel launch latency. The broader lesson for future optimization work: an algorithm's asymptotic complexity class doesn't tell you where time is actually spent. Our Phase-3 pdist GPU port projected a 25–35% end-to-end speedup on 4.7 h based on "clustering is 41% of wall time;" measurement showed scipy's linkage was only ∼12 s of that 138 s stage, so an infinite-speedup linkage couldn't exceed an 8% stage gain. The rest of clustering is VBxClustering-specific CPU work (PLDA re-scoring, AHC resolution search, twice-called `assign_embeddings`) that would need stage-level restructuring to move off CPU.

Negative result: segmentation step reduction. We tested reducing PyAnnote's default segmentation sliding-window overlap from 90% (`step=1.0 s` over a 10 s chunk) to 80% and 70% (`step=2.0 s`, `step=3.0 s`). The wall-time side behaved exactly as predicted: step=2.0 gave a 51% end-to-end speedup on the 2.2 h reference and step=3.0 a 67% speedup, because the segmentation model processes proportionally fewer chunks and the downstream embedding and clustering stages receive a smaller filtered set. DER on both configurations, however, regressed beyond the 1.0 pp opt-in gate (1.45 pp at step=2.0, 1.73 pp at step=3.0) with a speaker-count mismatch (3 → 2 on the reference file in both cases). The mechanism: PyAnnote's binarized segmentation derives each frame's label from the average of every overlapping chunk's posterior at that frame. At 90% overlap a 2 s turn by a minor speaker is covered by 20 chunks; at 70% overlap it's only 6, and if a few of those happen to fall near the model's less-accurate chunk edges the averaged posterior falls below the binarization threshold and the turn disappears entirely. The speaker is then dropped from the downstream clustering, which compounds into a full-pipeline speaker-count error. This failure mode is a calibration-at-training-time artefact—PyAnnote was trained with 90% overlap and doesn't extrapolate gracefully. A retraining-style fix is out of scope for this work, but the `segmentation_step` knob remains available for low-speaker-count inputs (interviews, monologues) where the short-turn- degradation pathway doesn't apply.

Availability. OpenTranscribe is available at https://github.com/davidamacey/OpenTranscribe under the AGPL-3.0 license. The PyAnnote optimization fork is at https://github.com/davidamacey/pyannote-audio (branch `gpu-optimizations`).

References

[1] SYSTRAN, "faster-whisper: Faster Whisper transcription with CTranslate2," https://github.com/SYSTRAN/faster-whisper, 2023, accessed: 2026-03-22.

[2] OpenNMT, "CTranslate2: Fast inference engine for transformer models," https://github.com/OpenNMT/CTranslate2, 2023, accessed: 2026-03-22.

[3] H. Bredin, "pyannote.audio 2.1 speaker diarization pipeline: principle, benchmark, and recipe," in *Proceedings of INTERSPEECH 2023*, 2023, pp. 1983–1987.

[4] Celery Project, "Celery: Distributed task queue," https://docs.celeryq.dev/, 2024, accessed: 2026-03-22.

[5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, vol. 202. PMLR, 2023, pp. 28 492–28 518. [Online]. Available: https://proceedings.mlr.press/v202/radford23a.html

[6] A. Baevski, Y. Zhou, A. Mohamed, and M. Auli, "wav2vec 2.0: A framework for self-supervised learning of speech representations," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33. Curran Associates, Inc., 2020, pp. 12 449–12 460. [Online]. Available: https://proceedings.neurips.cc/paper/2020/hash/92d1e1eb1cd6f9fba3227870bb6d7f07-Abstract.html

[7] M. Bain, J. Huh, T. Han, and A. Zisserman, "WhisperX: Time-accurate speech transcription of long-form audio," in *Proceedings of INTERSPEECH 2023*, 2023, pp. 4489–4493.

[8] P. Micikevicius, S. Narang, J. Alben, G. Diamos, E. Elsen, D. Garcia, B. Ginsburg, M. Houston, O. Kuchaiev, G. Venkatesh, and H. Wu, "Mixed precision training," in *Proceedings of the 6th International Conference on Learning Representations (ICLR)*, 2018. [Online]. Available: https://openreview.net/forum?id=r1gs9JgRZ

[9] H. Bredin, R. Yin, J. M. Coria, G. Gelly, P. Korshunov, M. Lavechin, D. Fustes, H. Titeux, W. Bouaziz, and M.-P. Gill, "pyannote.audio: neural building blocks for speaker diarization," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2020, pp. 7124–7128.

[10] H. Wang, C. Liang, S. Wang, Z. Chen, B. Zhang, X. Xiang, Y. Deng, and Y. Qian, "WeSpeaker: A research and production oriented speaker embedding learning toolkit," in *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2023.

[11] F. Landini, J. Profant, M. Diez, and L. Burget, "Bayesian HMM clustering of x-vector sequences (VBx) for speaker diarization," *Computer Speech & Language*, vol. 71, p. 101254, 2022.

[12] NVIDIA Corporation, "NVIDIA A100 tensor core GPU architecture," NVIDIA, Tech. Rep., 2020, whitepaper v1.0. [Online]. Available: https://www.nvidia.com/content/dam/en-zz/Solutions/Data-Center/nvidia-ampere-architecture-whitepaper.pdf

[13] ——, *CUDA C++ Programming Guide*, 2024, release 12.4. Accessed: 2026-03-22. [Online]. Available: https://docs.nvidia.com/cuda/cuda-c-programming-guide/

[14] Deepgram, "Deepgram: AI speech recognition and understanding," https://deepgram.com/, 2024, accessed: 2026-03-22.

[15] OpenAI, "GPT-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.

[16] AssemblyAI, "AssemblyAI: AI models to transcribe and understand speech," https://www.assemblyai.com/, 2024, accessed: 2026-03-22.

[17] Amazon Web Services, "Amazon Transcribe: Automatic speech recognition service," https://aws.amazon.com/transcribe/, 2024, accessed: 2026-03-22.